referenced in the `DbSet<>` properties on your app's `DbContext` and any linked types.

For the recipe app, EF Core builds a model of the `Recipe` class because it's exposed on the `AppDbContext` as a `DbSet<Recipe>`. Furthermore, EF Core loops through all the properties of `Recipe`, looking for types it doesn't know about, and adds them to its internal model. In the app, the `Ingredients` collection on `Recipe` exposes the `Ingredient` entity as an `ICollection<Ingredient>`, so EF Core models the entity appropriately.

EF Core maps each entity to a table in the database, but it also maps the relationships between the entities. Each recipe can have many ingredients, but each ingredient (which has a name, quantity, and unit) belongs to one recipe, so this is a many-to-one relationship. EF Core uses that knowledge to correctly model the equivalent many-to-one database structure.

NOTE Two different recipes, such as fish pie and lemon chicken, may use an ingredient that has both the same name and quantity, such as the juice of one lemon, but they're fundamentally two different instances. If you update the lemon chicken recipe to use two lemons, you wouldn't want this change to automatically update the fish pie recipe to use two lemons, too!

EF Core uses the internal model it builds when interacting with the database to ensure that it builds the correct SQL to create, read, update, and delete entities.

Right—it's about time for some code! In section 12.2, you'll start building the recipe app. You'll see how to add EF Core to an ASP.NET Core application, configure a database provider, and design your application's data model.